

DLIC_HW9 CONV

系級:資訊 114 甲班 學號:E34106010 姓名:黃偉峰

1. RTL 的模擬結果截圖

```
START!!! Simulation Start .....
```

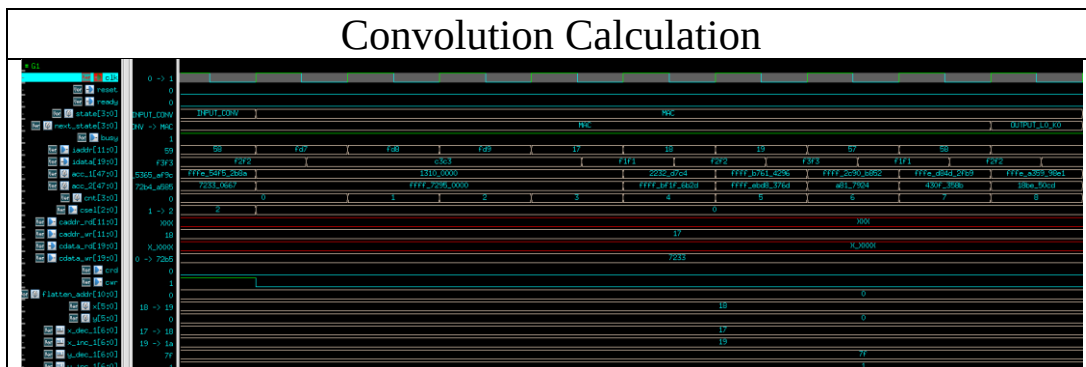
```
Layer 0 (Convolutional Output) with Kernel 0 is correct !
Layer 0 (Convolutional Output) with Kernel 1 is correct!
Layer 1 (Max-pooling Output) with Kernel 0 is correct!
Layer 1 (Max-pooling Output) with Kernel 1 is correct!
Layer 2 (Flatten Output) is correct!
```

```
----- S U M M A R Y -----
```

```
Congratulations! Layer 0 data have been generated successfully! The result is PASS!!
Congratulations! Layer 1 data have been generated successfully! The result is PASS!!
Congratulations! Layer 2 data have been generated successfully! The result is PASS!!
```

```
$finish called from file "testfixture.v", line 278.
$finish at simulation time          1249400000
      V C S   S i m u l a t i o n   R e p o r t
Time: 1249400000 ps
CPU Time:      1.070 seconds;      Data structure size:   0.2Mb
Sun May 18 19:49:06 2025
CPU time: .607 seconds to compile + .634 seconds to elab + .490 seconds to link + 1.122 seconds in simulation
```

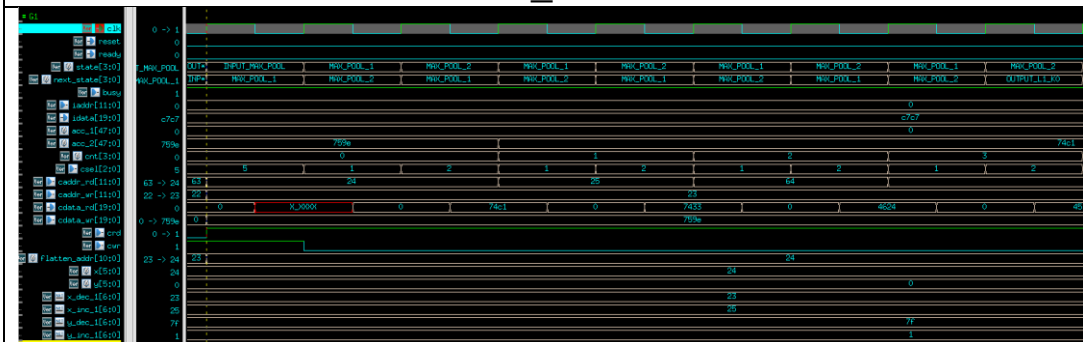
2. 解釋波型



在 INPUT_CONV state 預先將 iaddr 提供給 tb，以便在下一個 clock 時得到該位置的值，將 Convolution 的計算視為 9 個 MAC 運算的操作並且在 accumulator 內初始化 bias 值在內，每個 cycle 都先要下一個位置的值並且計算目前位置的值，經過 9 個 cycle MAC 計算後得到 Convolution 的值，以上過程中兩份乘法器同時操作

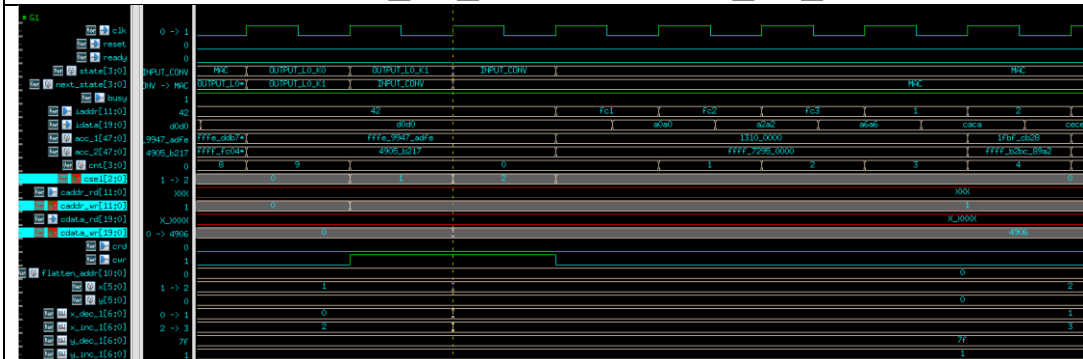
kernel0、kernel1 的 convolution 計算。

MAX_POOL



整張圖片的 Convolution 計算完後，開始計算 MAX_POOL，在 INPUT_MAX_POOL state 下，提前放置 addr，以便在下個 cycle 隨即得到該值拿來作比大小的操作，MAX_POOL_1 state 接收 L0MEM0 的值，MAX_POOL_2 state 則接收 L0MEM1 的值，在這兩個 state 下將新的值跟已有的值比大小，經過 8 個 cycle 比較出兩者的最大值。

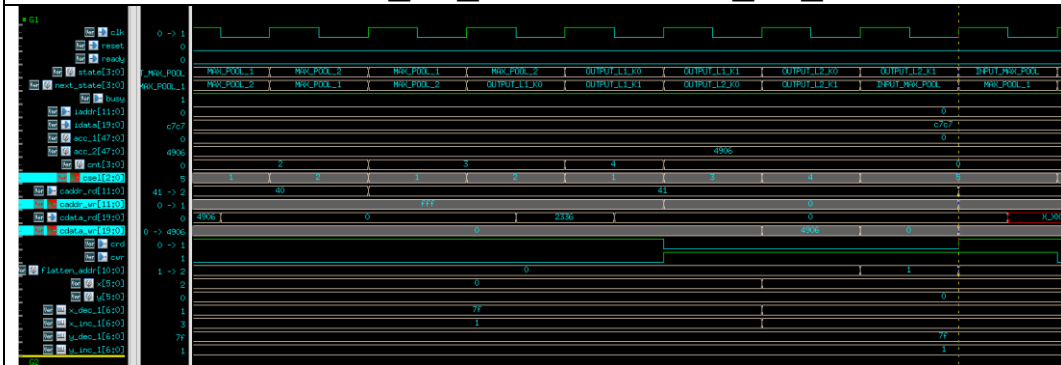
OUTPUT_L0_K0 / OUTPUT_L0_K1



Convolution 過後的結果透過 OUTPUT_L0_K0 state 將 kernel0 的值輸出到 LOMEM0 當中，OUTPUT_L0_K1 state 將 kernel1 的值輸出到 LOMEM1 當中。

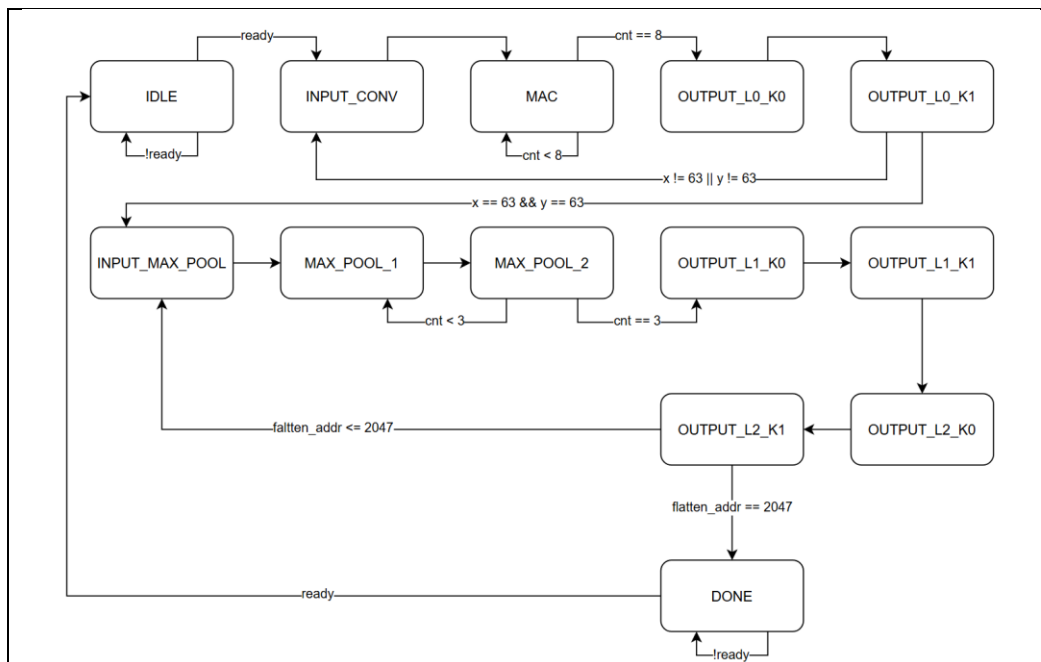
OUTPUT_L1_K0 / OUTPUT_L1_K1

OUTPUT_L2_K0 / OUTPUT_L2_K1



Max Pool 計算後得到兩個 kernel 各別的最大值後，隨即在 OUTPUT_L1_K0 / OUTPUT_L1_K1 state 下各別將兩 kernel 下的最大值輸出到 L1MEM0、L1MEM1，並且也一起在 MaxPool 計算出來後將值做 flatten 操作，將兩 kernel 的值透過 OUTPUT_L2_K0 / OUTPUT_L2_K1 state，分別輸入進去 L2MEM 當中。

3. 畫出狀態機



4. 學到什麼？

這次實作讓我比較有感的是，原本在 paper 或課本上看起來很直覺的 CNN 流程（像是 convolution → max pooling → flatten），其實要用硬體一個一個步驟實作起來還是滿麻煩的。像是地址怎麼排、邊界怎麼處理、資料怎麼存來存去，都要自己想清楚。我也更熟悉了狀態機怎麼規劃、怎麼讓資料在每個階段順順地跑完，算是把「看得懂」變成「做得出來」。雖然東西不算太難，但實際寫過一次之後對整個 CNN 運作流程更有感，也比較知道如果要做更大的網路，底層大概會長什麼樣子。